

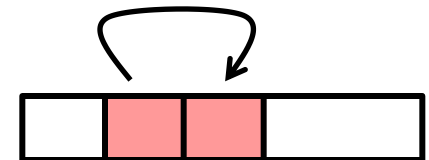
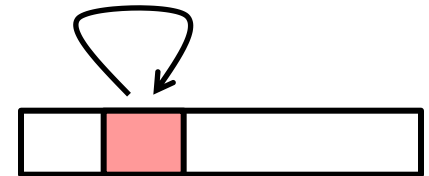
Memory Hierarchy (II)

Outline

- Locality
- The memory hierarchy
- Cache Memory
- Suggested Reading: 6.2, 6.3, 6.4

Locality

- **Principle of Locality:** Programs tend to use data and instructions with addresses **near** or **equal** to those they have used recently
 - Temporal locality
 - Recently referenced items are likely to be referenced again in the near future
 - Spatial locality
 - Items with nearby addresses tend to be referenced close together in time



Locality

- All levels of modern computer systems are designed to exploit **locality**
 - Hardware
 - Cache memory (to speed up main memory accesses)
 - Operating systems
 - Use main memory to speed up virtual address space accesses
 - Use main memory to speed up disk file accesses
 - Application programs
 - Web browsers exploit temporal locality by caching recently referenced documents on a local disk

Locality

```
int sumvec(int v[N])
{
    int i, sum = 0 ;

    for (i = 0 ; i < N ; i++)
        sum += v[i] ;
    return sum ;
}
```

Address	0	4	8	12	16	20	24	28
Contents	v ₀	v ₁	v ₂	v ₃	v ₄	v ₅	v ₆	v ₇
Access order	1	2	3	4	5	6	7	8

Locality

- Locality in the example
 - sum, i: temporal locality
 - v: spatial locality
 - Stride-1 reference pattern
- Stride-k reference pattern
 - Visiting every k-th element of a contiguous vector
 - As the stride increases, the spatial locality decreases

Stride-1 reference pattern

```
int sumarrayrows(int a[M][N]) //M=2,N=3
{
    int i, j, sum = 0 ;

    for (i = 0 ; i < M ; i++)
        for ( j = 0 ; j < N ; j++ )
            sum += a[i][j] ;
    return sum ;
}
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	2	3	4	5	6

Stride-N reference pattern

```
int sumarraycols(int a[M][N]) //M=2,N=3
{
    int i, j, sum = 0 ;

    for (j = 0 ; j < N ; j++)
        for ( i = 0 ; i < M ; i++ )
            sum += a[i][j] ;
    return sum ;
}
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	3	5	2	4	6

Locality

- Locality of the instruction fetch
 - Spatial locality
 - In most cases, programs are executed in **sequential** order
 - Temporal locality
 - Instructions in **loops** may be executed many times

Locality

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

- Data references

- Reference array **a** elements in succession
- Reference variable **sum** each iteration

Spatial locality

Temporal locality

- Instruction references

- Reference instructions in **sequence**.
- Cycle through **loop** repeatedly

Spatial locality

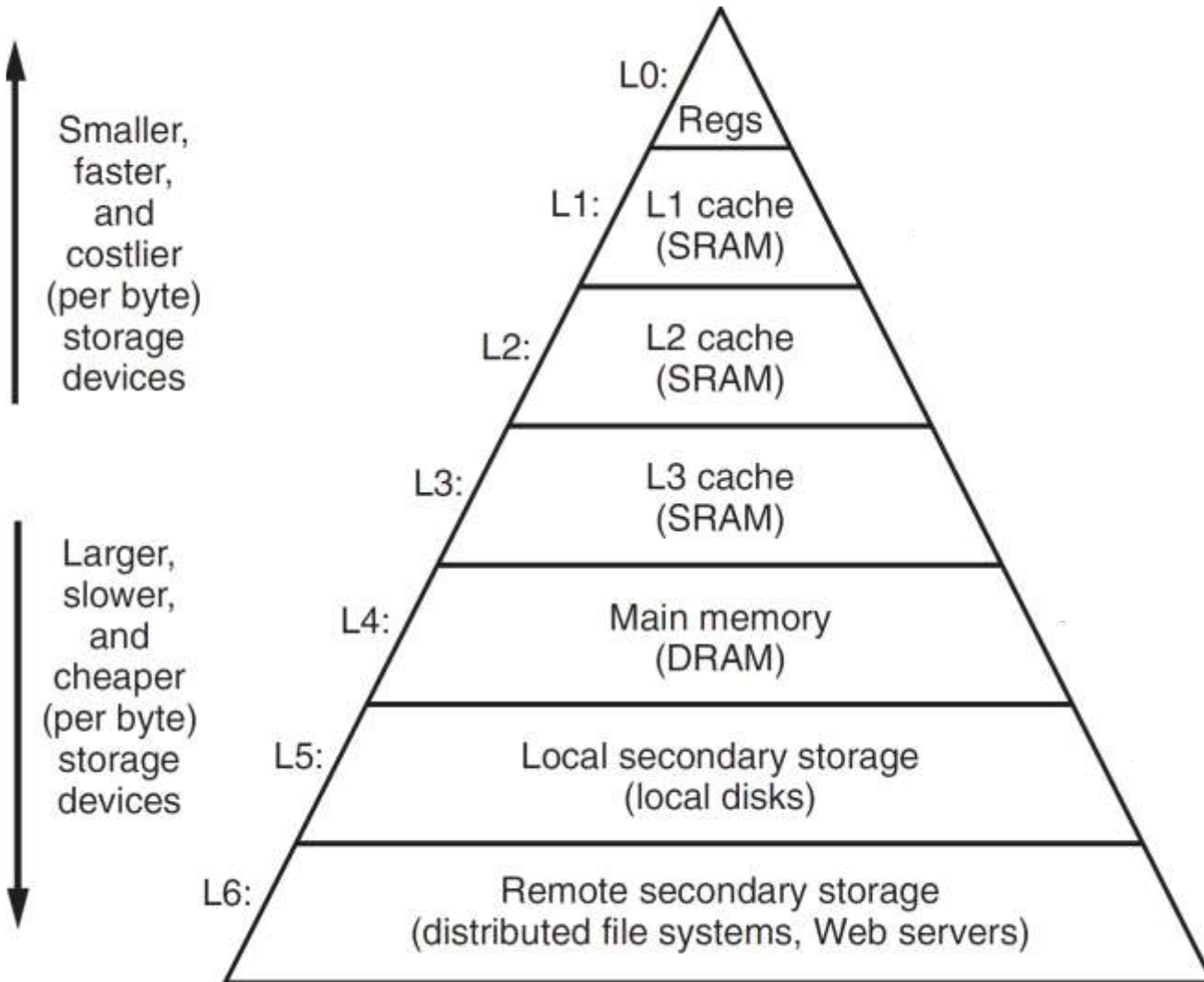
Temporal locality

Memory Hierarchy

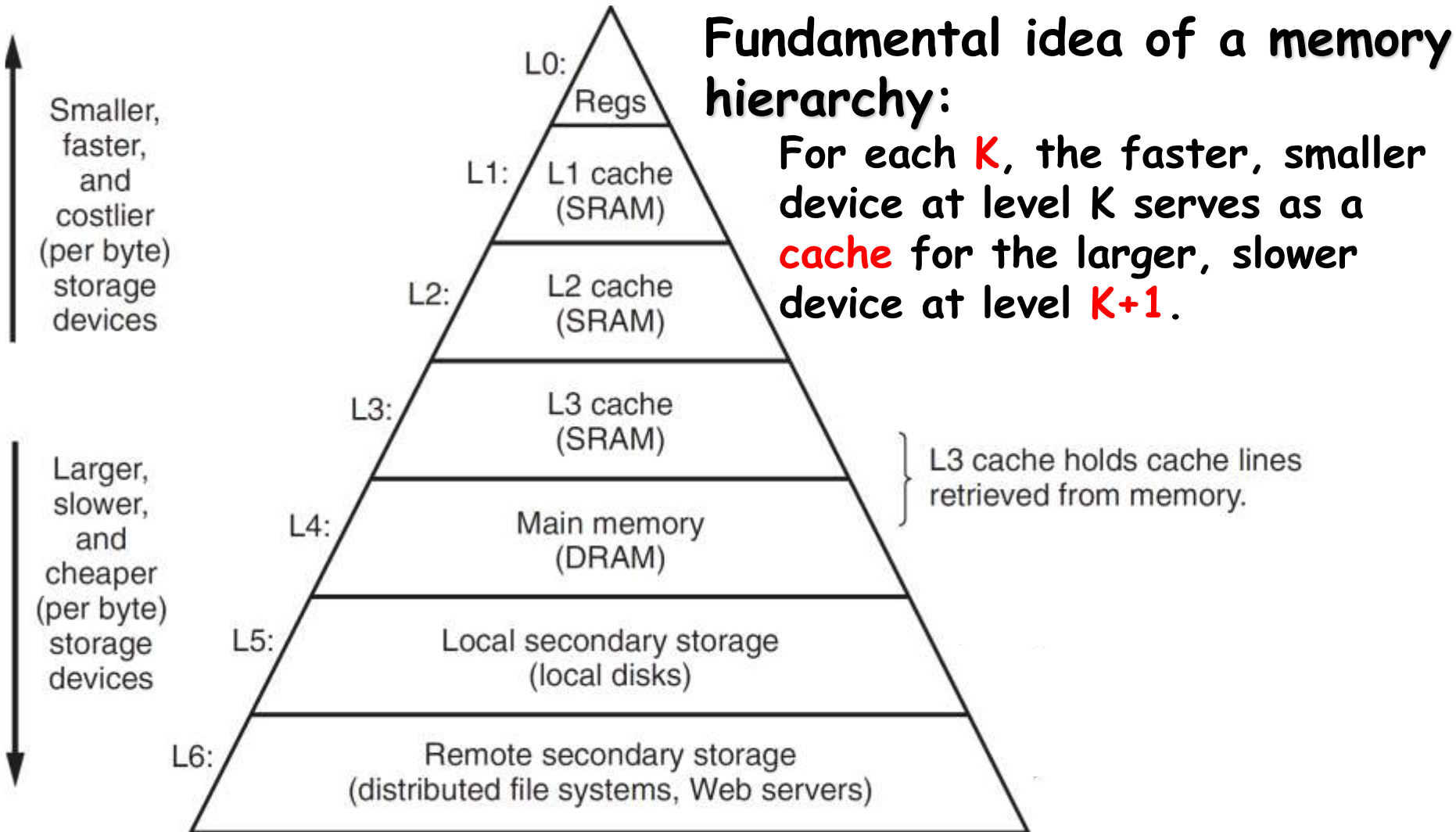
Memory Hierarchy

- Fundamental properties of storage technology and computer software
 - Different storage technologies have widely different **access times**
 - Faster technologies **cost** more per byte than slower ones and have less **capacity**
 - The gap between CPU and main memory speed is widening
 - Well-written programs tend to exhibit good **locality**

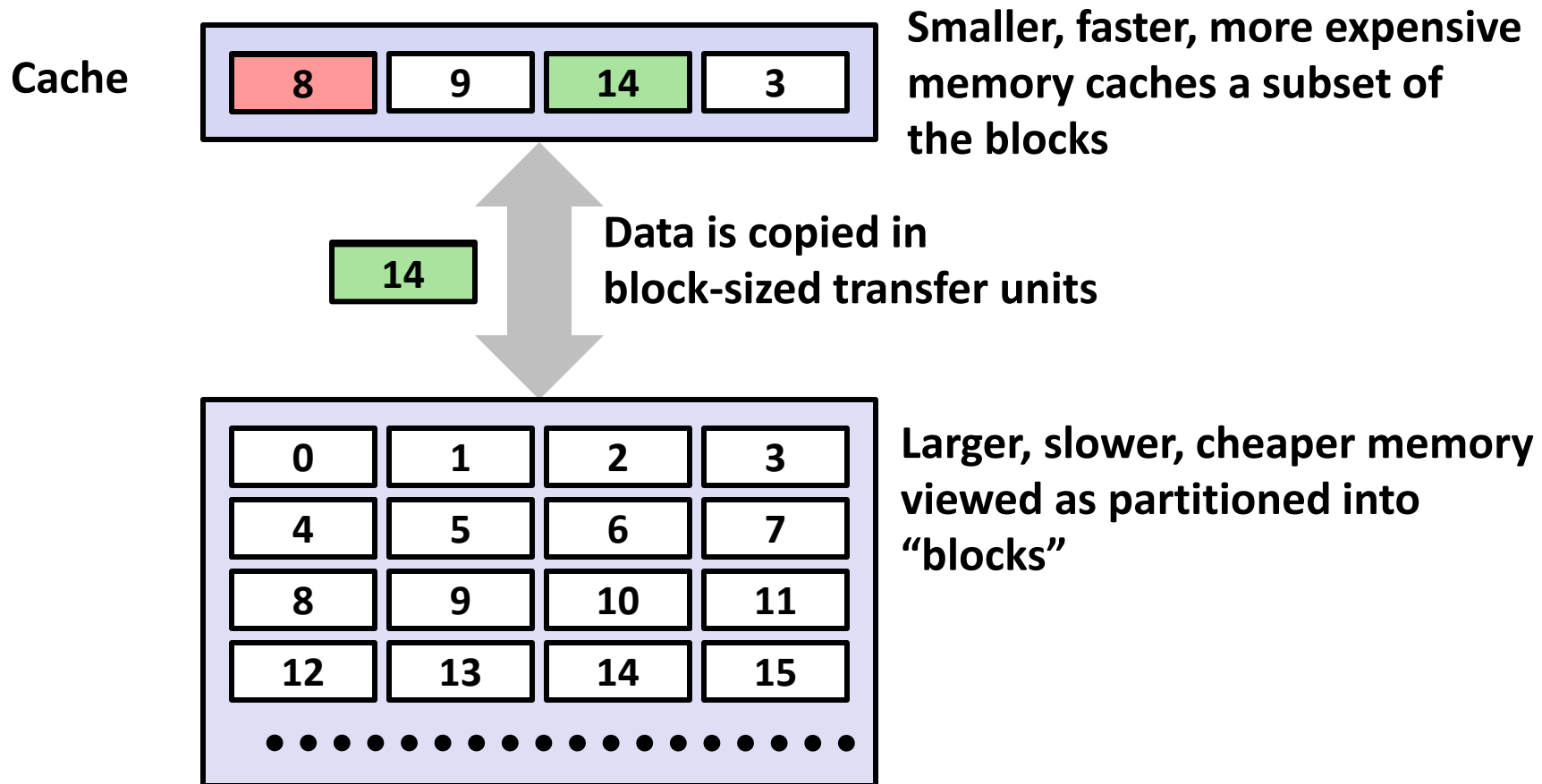
An example memory hierarchy



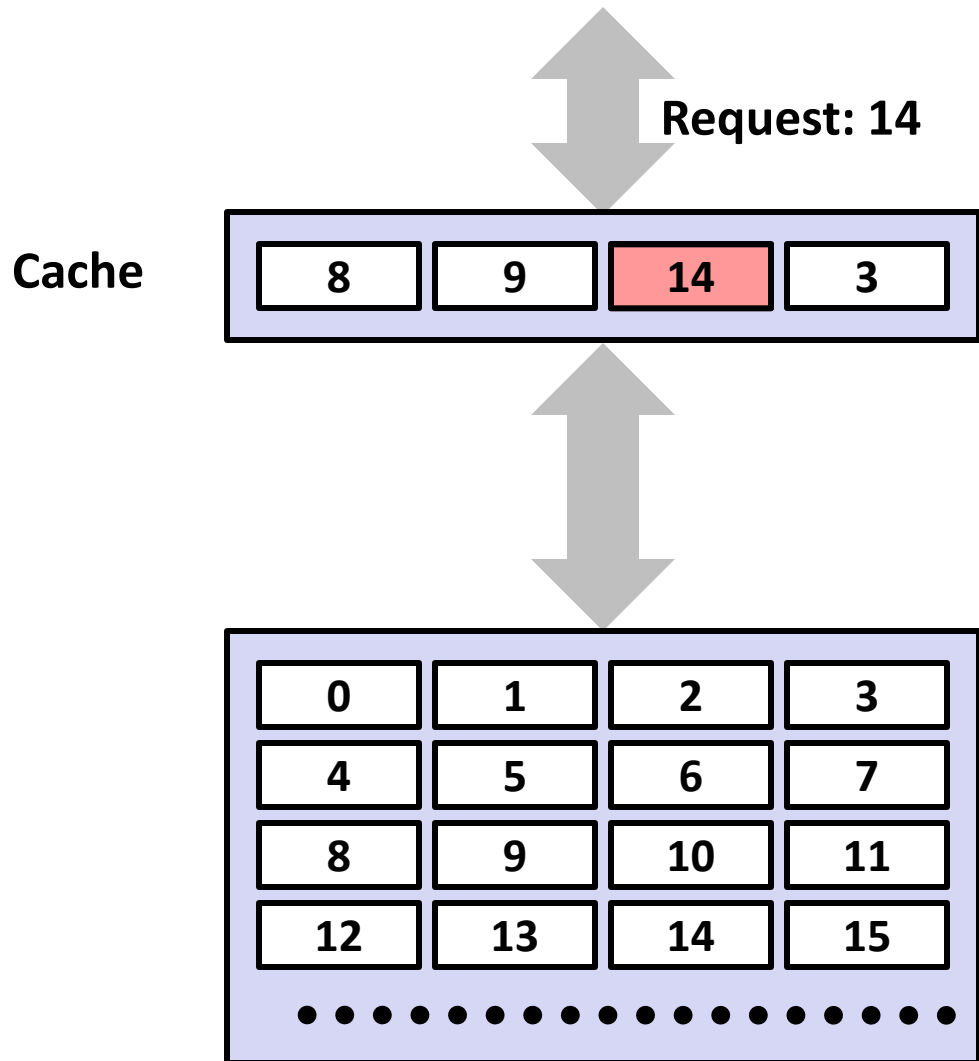
Caches



General Cache Concepts



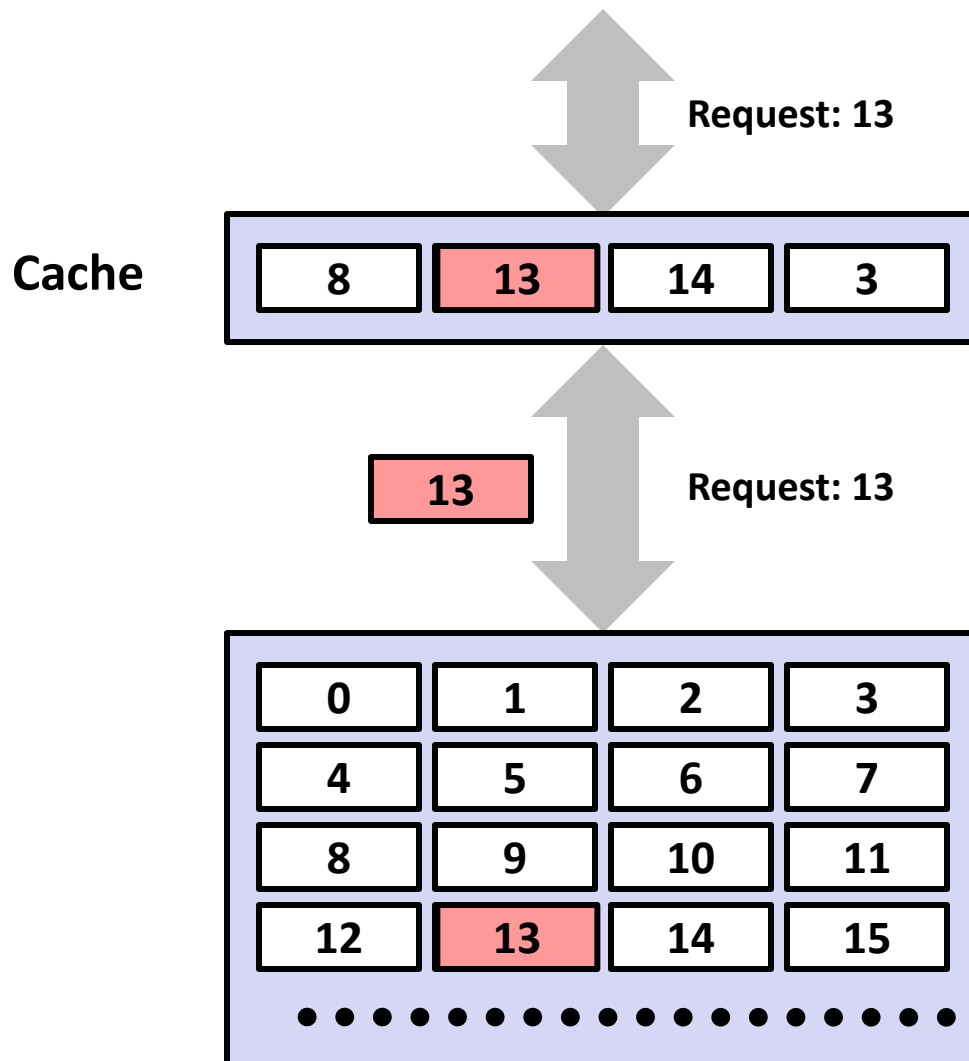
General Cache Concepts: Hit



Data in block b is needed

Block b is in cache:
Hit!

General Cache Concepts: Miss



Data in block b is needed

Block b is not in cache:
Miss!

Block b is fetched from memory

Block b is stored in cache

- **Placement policy:**
determines where b goes
- **Replacement policy:**
determines which block gets evicted (victim)

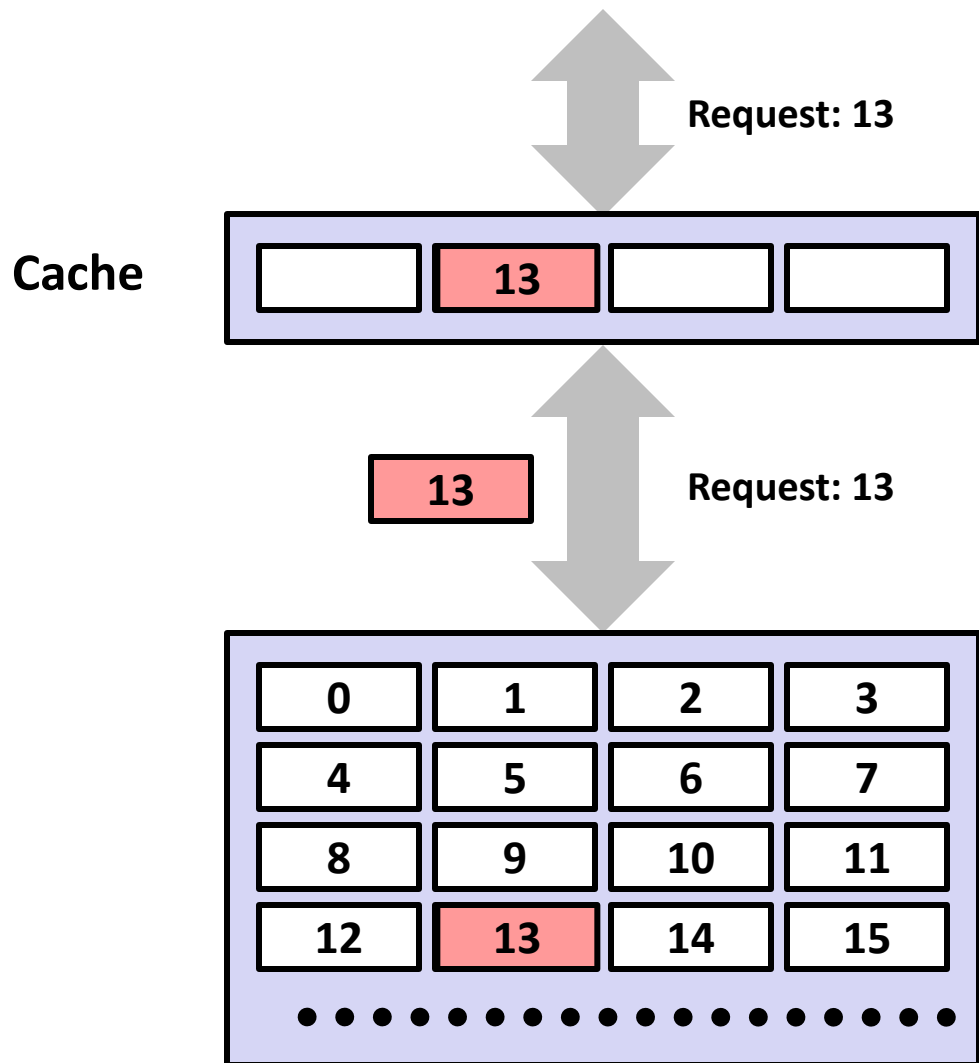
Types of Cache Misses

- Cold (compulsory) miss
 - Cold misses occur because the cache is **empty**.
- Capacity miss
 - Occurs when the set of active cache blocks (**working set**) is larger than the cache.

Types of Cache Misses

- Conflict miss
 - Most caches limit blocks at level $k+1$ to a small subset (sometimes a singleton) of the block **positions** at level k .
 - e.g. Block i at level $k+1$ must be placed in block $(i \bmod 4)$ at level k .
 - Conflict misses occur when the level k cache is large enough, but multiple data objects all map to the same level k block.
 - e.g. Referencing blocks 9, 13, 9, 13, 9, 13, ... would miss every time.

General Cache Concepts: Miss



Data in block 13 is needed

*Block 13 is not in cache:
Miss!*

*Block 13 is fetched from
memory*

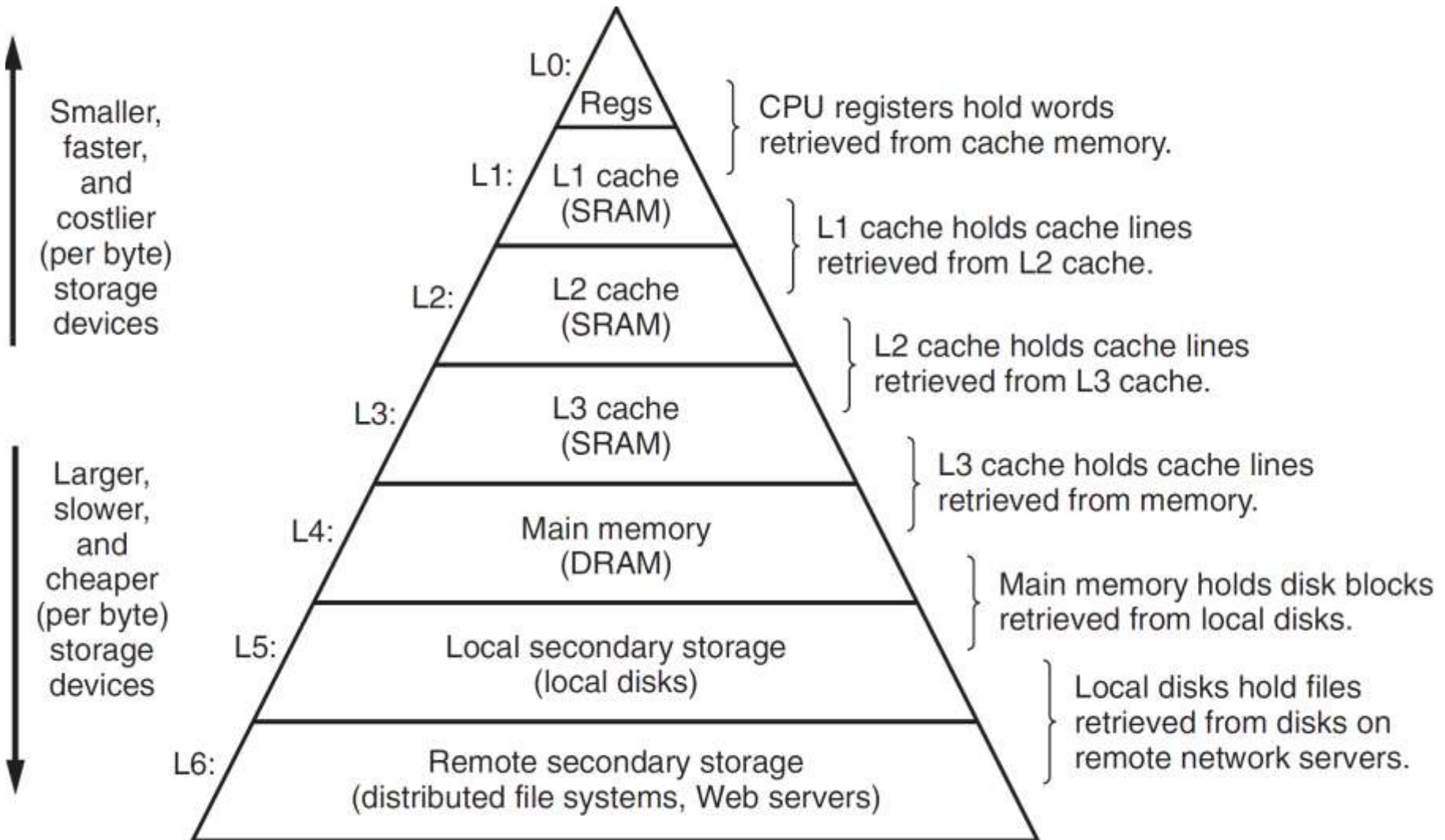
Block 13 is stored in cache

Caches

- Why do memory hierarchies work?
 - Because of **locality**, programs tend to access the data at level k more often than they access the data at level $k+1$.
 - Thus, the storage at level $k+1$ can be slower, and thus larger and cheaper per bit.

Big Idea: The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.

An example memory hierarchy



Examples of Caching in the Hierarchy

Cache Type	What is Cached?	Where is it Cached?	Latency (cycles)	Managed By
Registers	4-8 bytes words	CPU core	0	Compiler
TLB	Address translations	On-Chip TLB	0	Hardware
L1 cache	64-bytes block	On-Chip L1	4	Hardware
L2 cache	64-bytes block	On-Chip L2	10	Hardware
L3 cache	64-bytes block	On-Chip L3	50	Hardware
Virtual Memory	4-KB page	Main memory	200	Hardware + OS
Buffer cache	Parts of files	Main memory	200	OS
Disk cache	Disk sectors	Disk controller	100,000	Disk firmware
Network buffer cache	Parts of files	Local disk	10,000,000	AFS/NFS client
Browser cache	Web pages	Local disk	10,000,000	Web browser
Web cache	Web pages	Remote server disks	1,000,000,000	Web proxy server

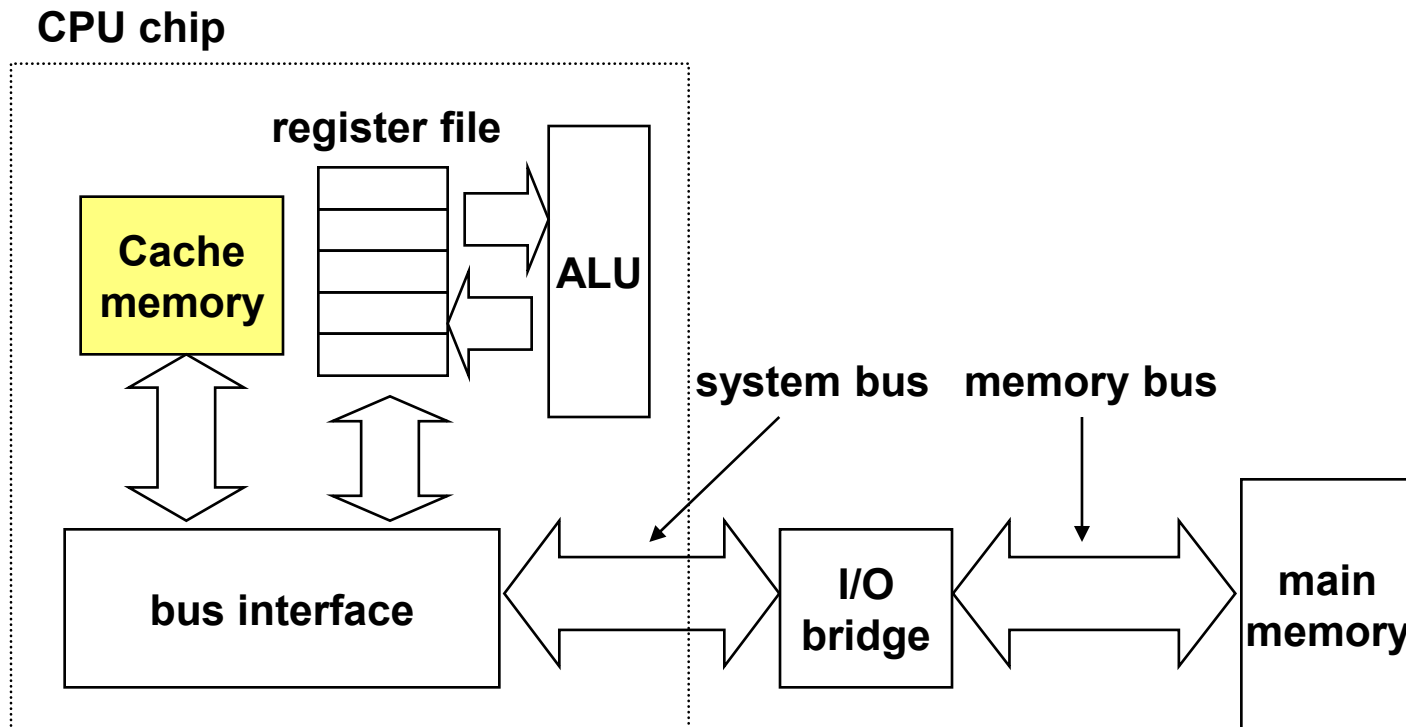
Cache Memory

Cache Memory

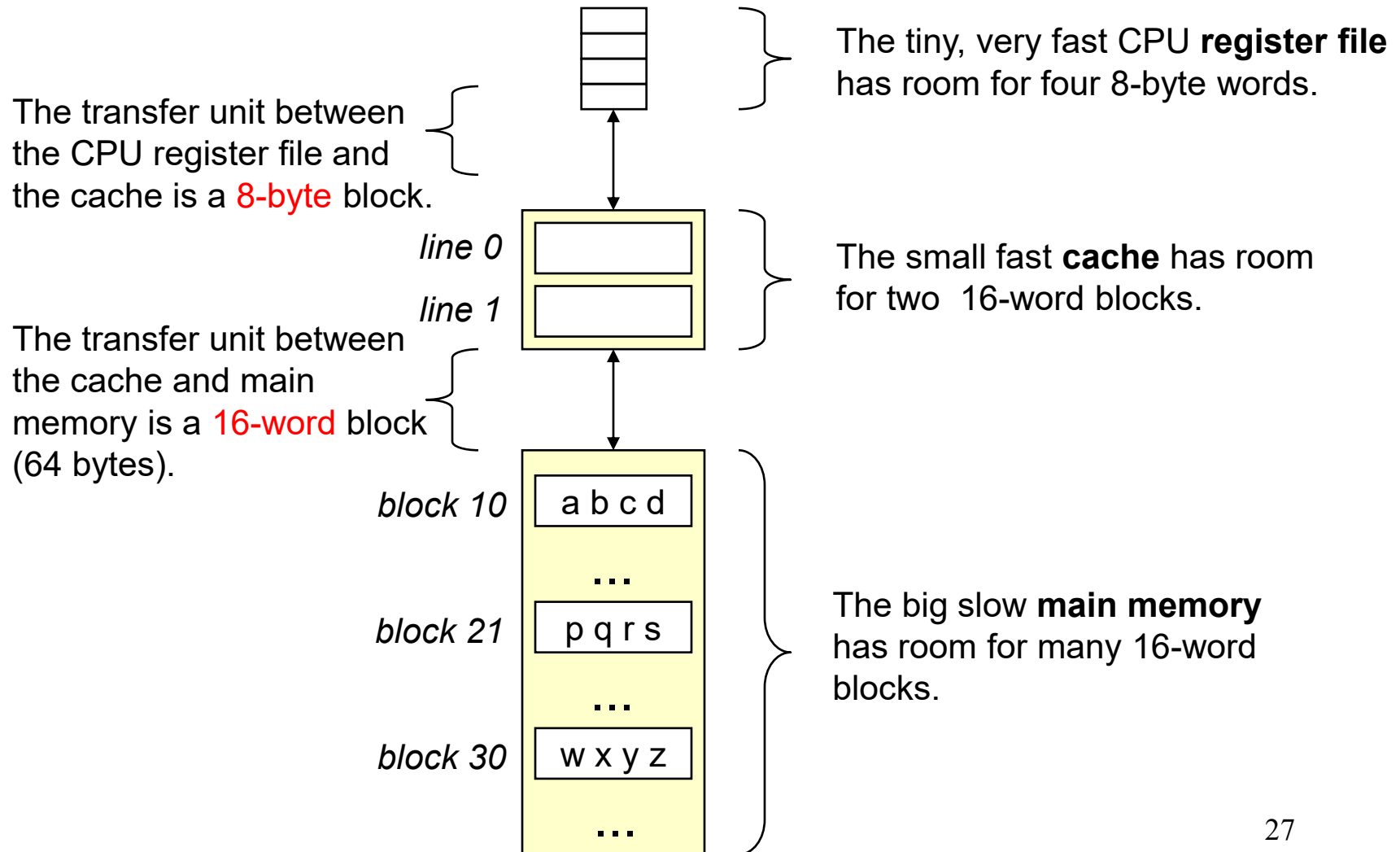
- History
 - At very beginning, 3 levels
 - Registers, main memory, disk storage
 - 10+ years later, 4 levels
 - Register, cache, main memory, disk storage
 - Modern processor, 6 levels
 - Registers, SRAM L1, L2, L3 cache, main DRAM memory, disk storage
 - Cache memories
 - small, fast SRAM-based memories
 - managed by hardware automatically

Cache Memory

- Hold frequently accessed blocks of main memory in caches
- CPU looks first for data in L1, then in L2, then in L3, then in main memory



Inserting a cache between the CPU and main memory



Memory Accessing

- For a memory accessing instruction
 - `movq A %rax`
- Access cache by *A* directly
- If cache hit
 - get the value from the cache
- Otherwise,
 - cache miss handling
 - get the value

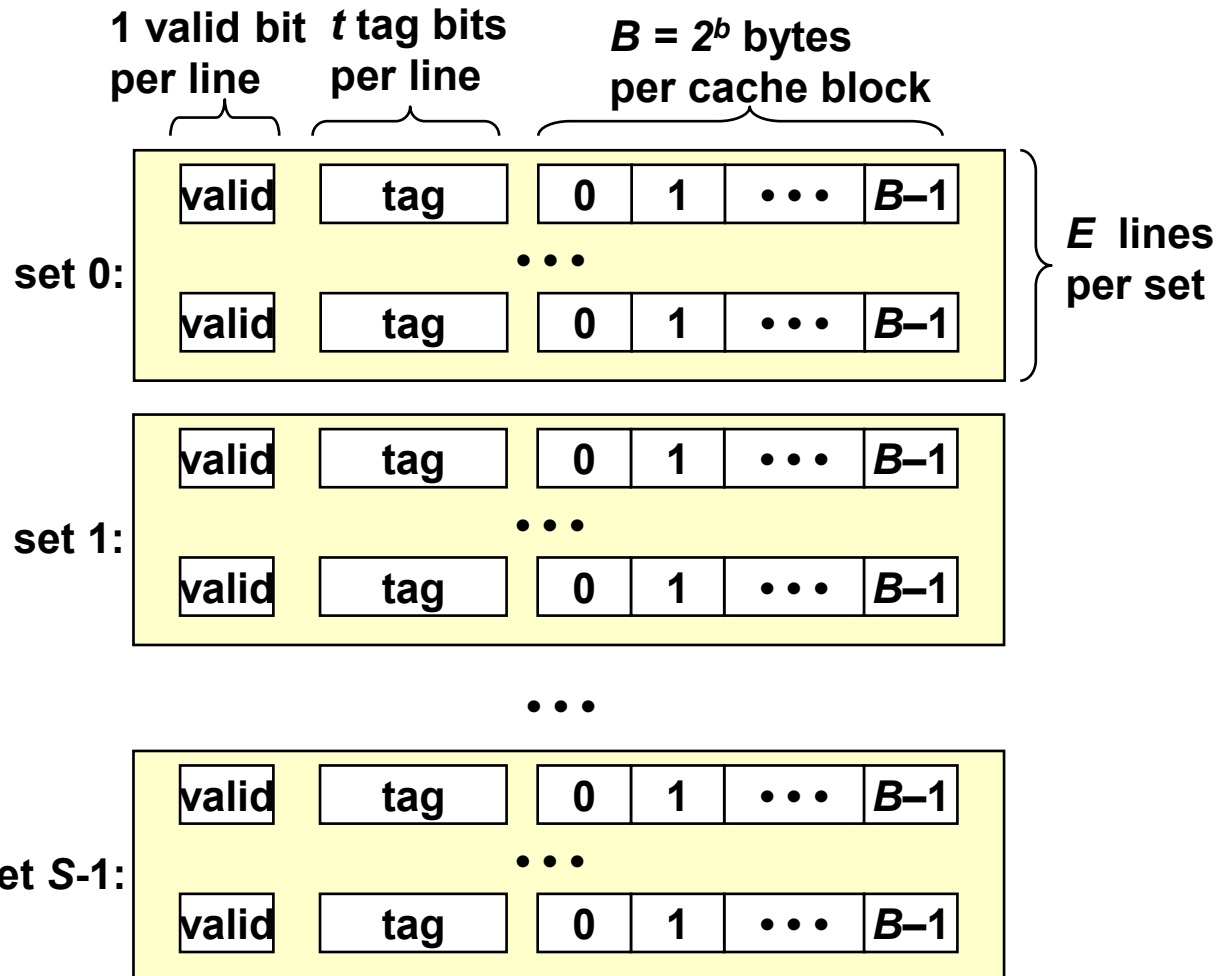
Generic Cache Memory Organization

Cache is an array of sets.

Each set contains one or more lines.

Each line holds a block of data.

$S = 2^s$ sets



Cache Memory

Fundamental parameters	
Parameters	Descriptions
$S = 2^s$	Number of sets
E	Number of lines per set
$B = 2^b$	Block size(bytes)
$m = \log_2(M)$	Number of physical(main memory) address bits

Cache Memory

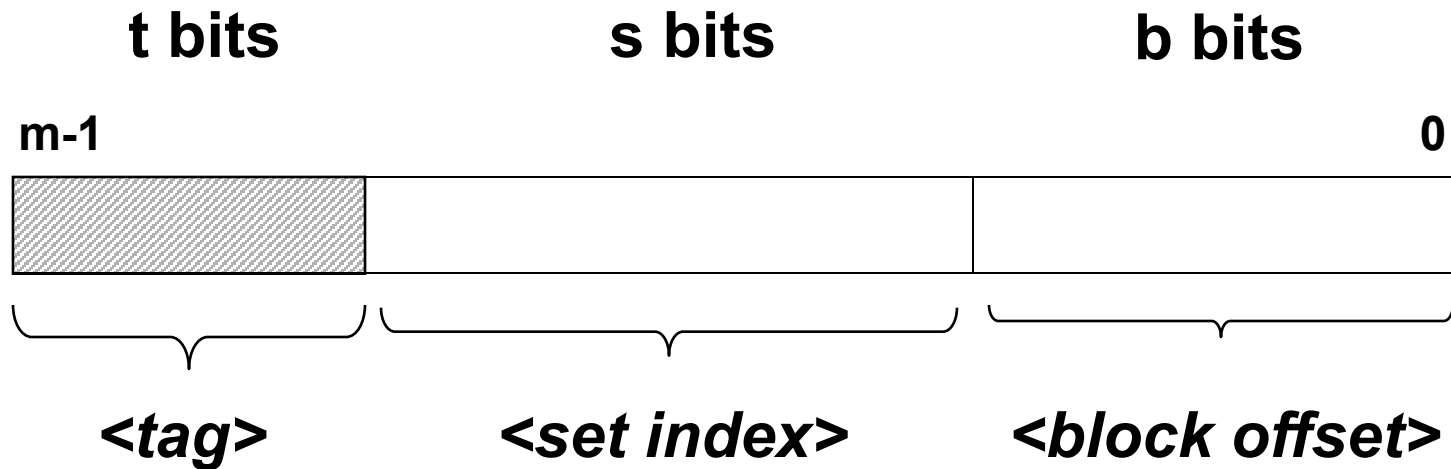
Derived quantities	
Parameters	Descriptions
$M=2^m$	Maximum number of unique memory address
$s=\log_2(S)$	Number of set index bits
$b=\log_2(B)$	Number of block offset bits
$t=m-(s+b)$	Number of tag bits
$C=B \times E \times S$	Cache size (bytes) not including overhead such as the valid and tag bits

Addressing caches

Physical Address A:

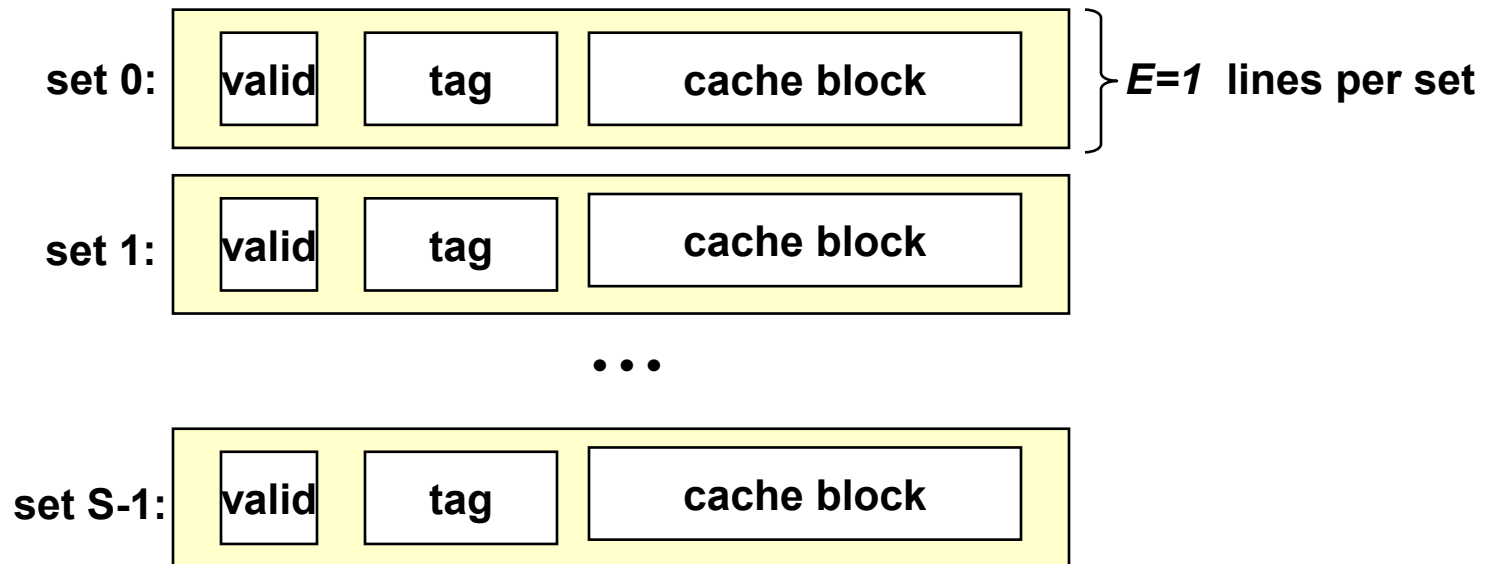


Split into 3 parts:



Direct-mapped cache

- Simplest kind of cache
- Characterized by exactly one line per set.

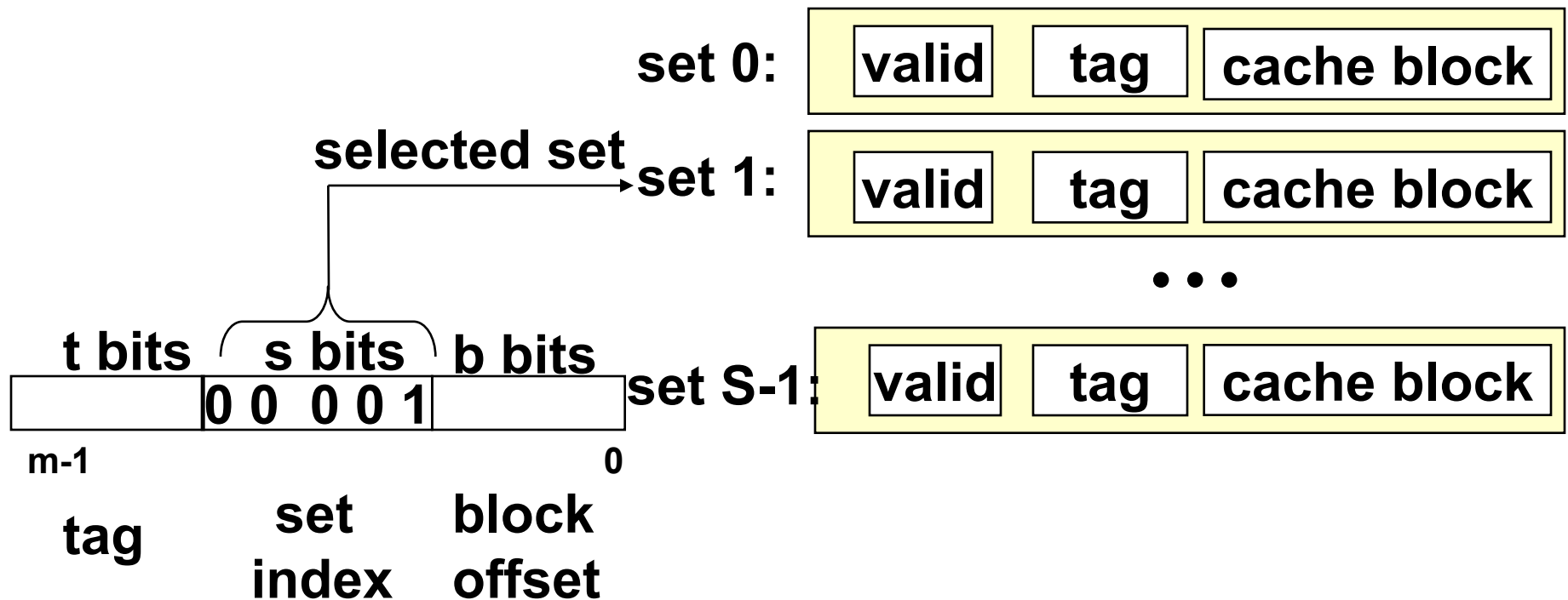


Accessing Direct-Mapped Caches

- Three steps
 - Set selection
 - Line matching
 - Word extraction

Set selection

- Use the set index bits to determine the set of interest

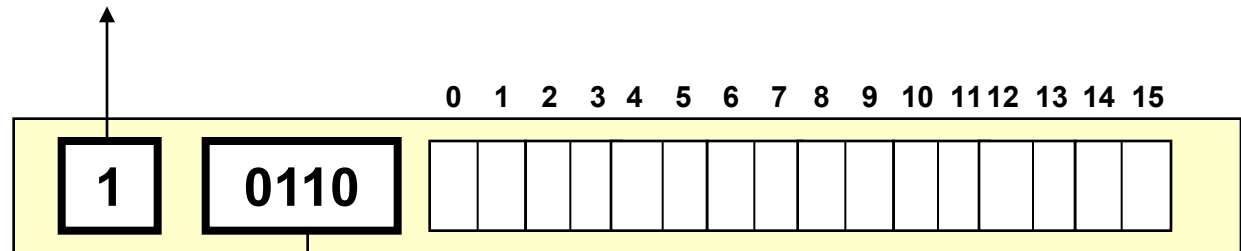


Line matching

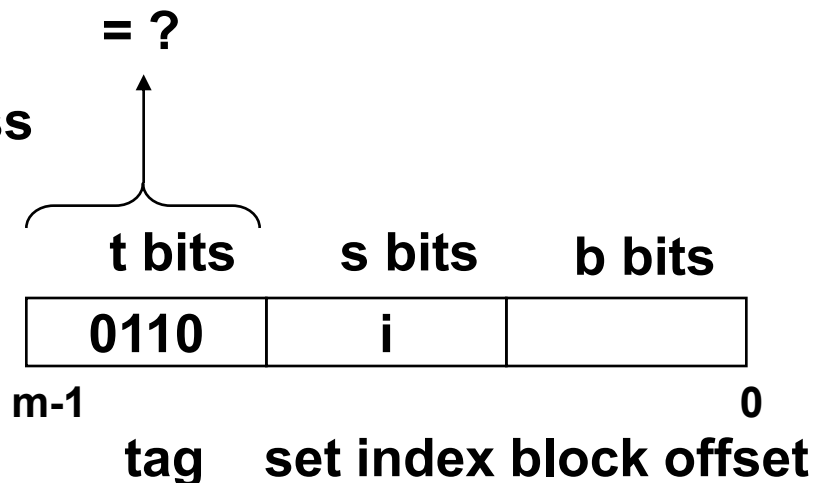
Find a valid line in the selected set with a matching tag

=1? (1) The valid bit must be set

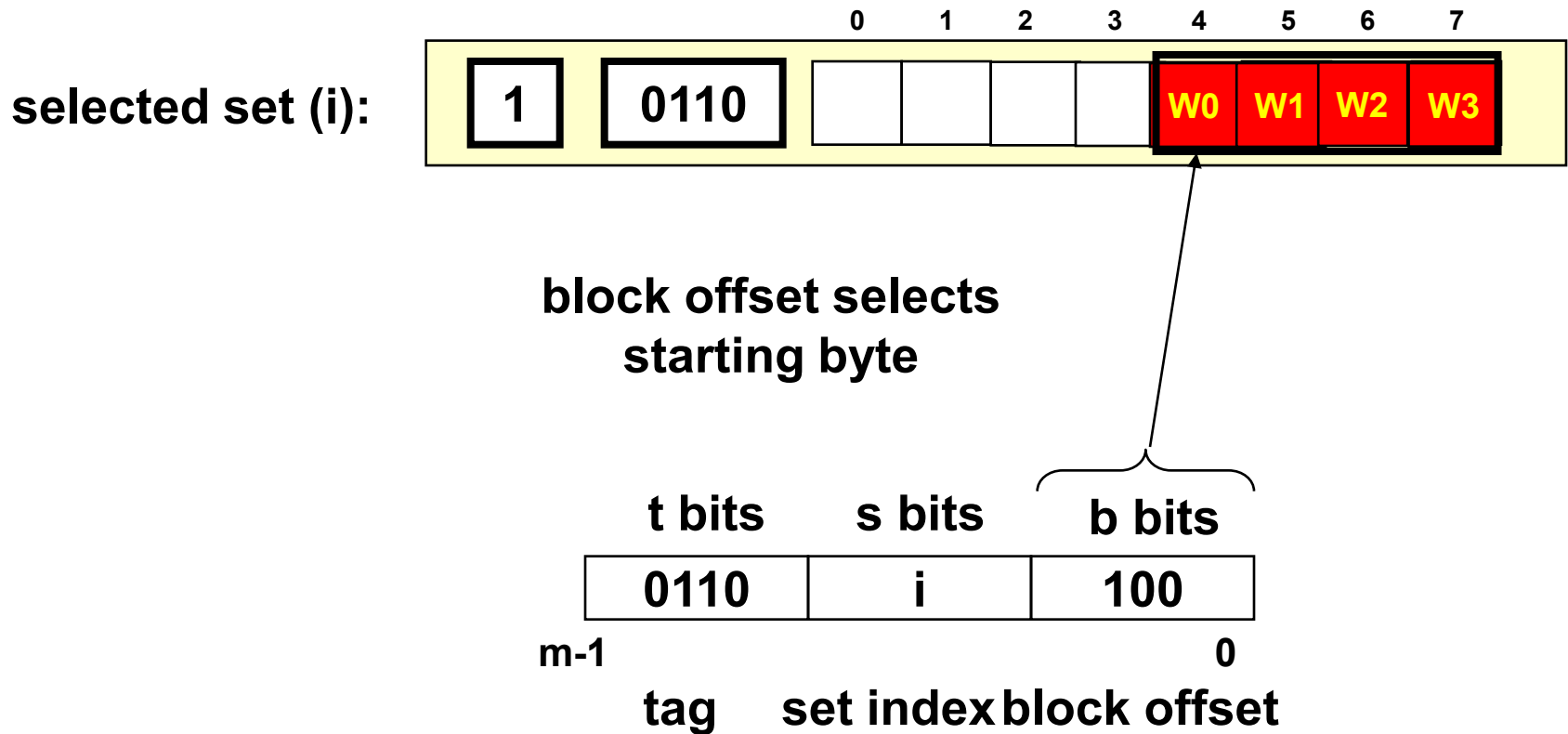
selected set (i):



(2) The tag bits in the cache line must match the tag bits in the address

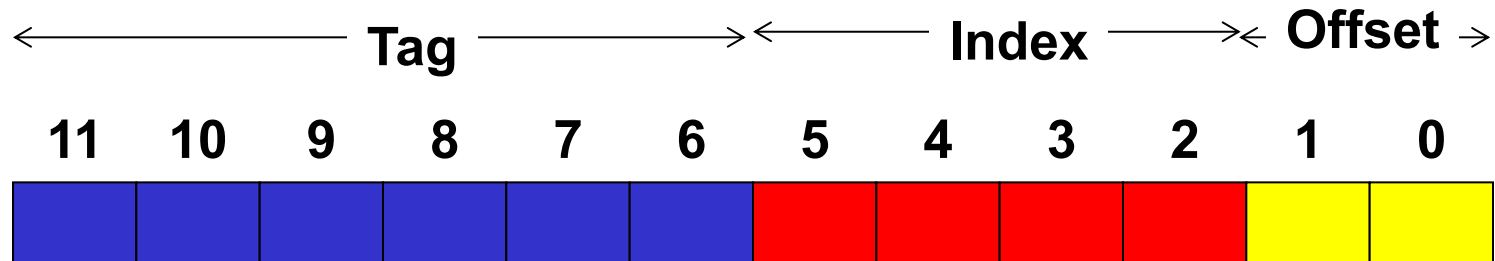


Word Extraction



Simple Memory System Cache

- Cache
 - 16 lines
 - 4-byte line size
 - Direct mapped



Simple Memory System Cache

Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
1	15	0	-	-	-	-
2	1B	1	00	02	04	08
3	36	0	-	-	-	-
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	-	-	-	-
7	16	1	11	C2	DF	03

Simple Memory System Cache

Idx	Tag	Valid	B0	B1	B2	B3
8	24	1	3A	00	51	89
9	2D	0	-	-	-	-
A	2D	1	93	15	DA	3B
B	0B	0	-	-	-	-
C	12	0	-	-	-	-
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	-	-	-	-

Address Translation Example

Address: 0x354

Tag						Index			Offset		
11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	0	1	0	1	0	1	0	0

Offset: 0x0 Index: 0x05 Tag: 0x0D

Hit? Yes Byte: 0x36

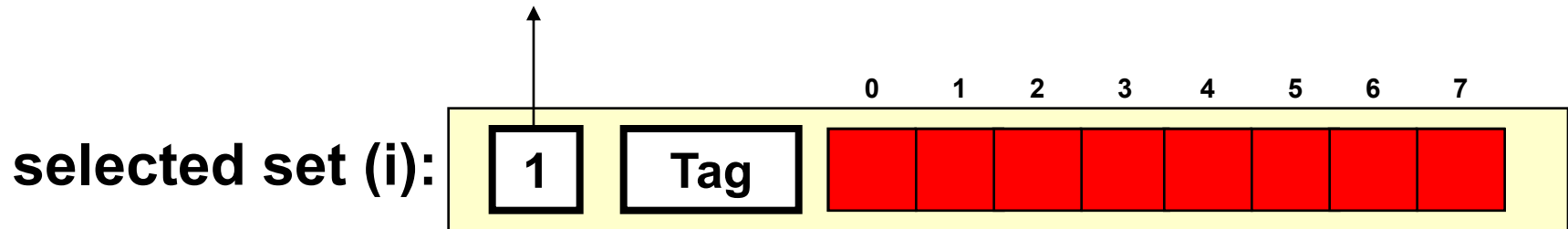
Idx	Tag	Valid	B0	B1	B2	B3
5	0D	1	36	72	F0	1D

Line Replacement on Misses

- Check the cache line of the set indicated by the set index bits
 - If the cache line valid, it must be evicted
- Retrieve the requested block from the next level
 - How to get the block ?
- Current line is replaced by the newly fetched line

Check the cache line

=1? If valid bit is set, evict the line



Get the Address of the Starting Byte

- Consider memory address looks like the following



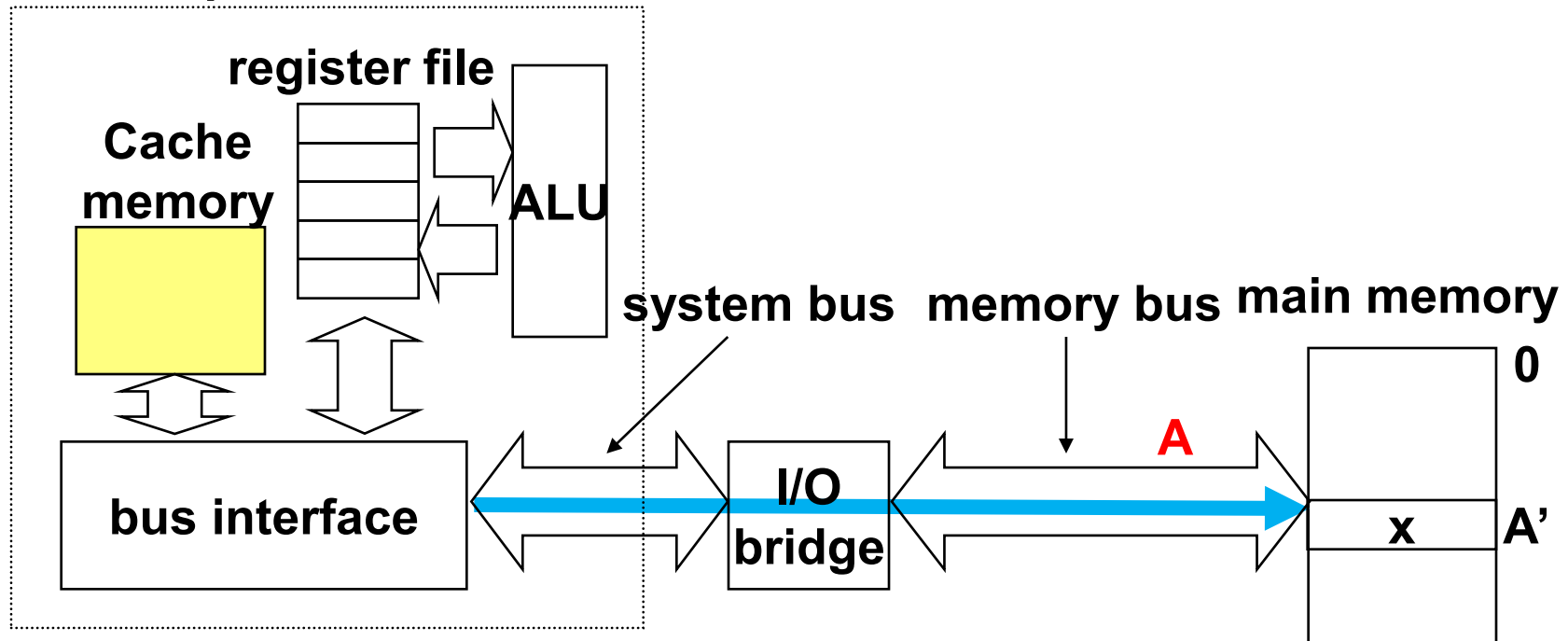
- Clear the last bits and get the address A



Read the Block from the Memory

Put A on the Bus (A is $A'000$)

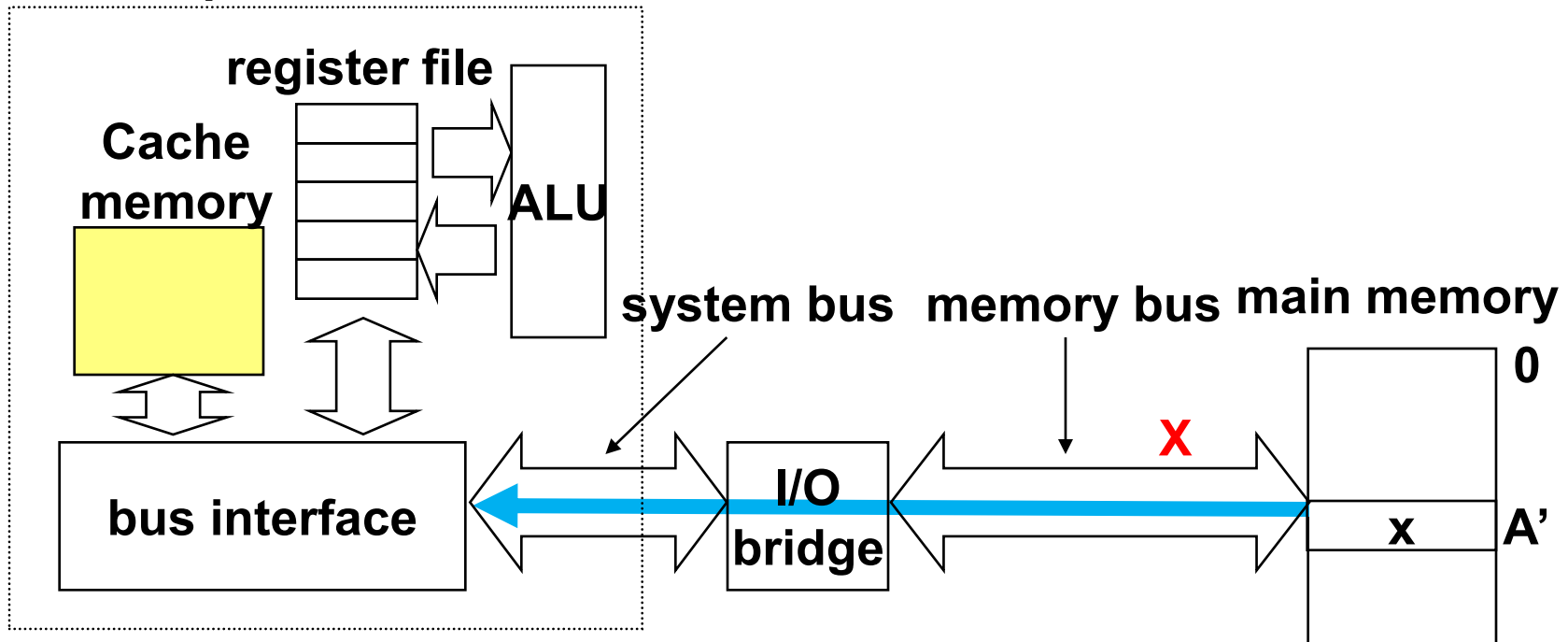
CPU chip



Read the Block from the Memory

Main memory reads **A'** from the memory bus, retrieves 8 bytes **x**, and places it on the bus

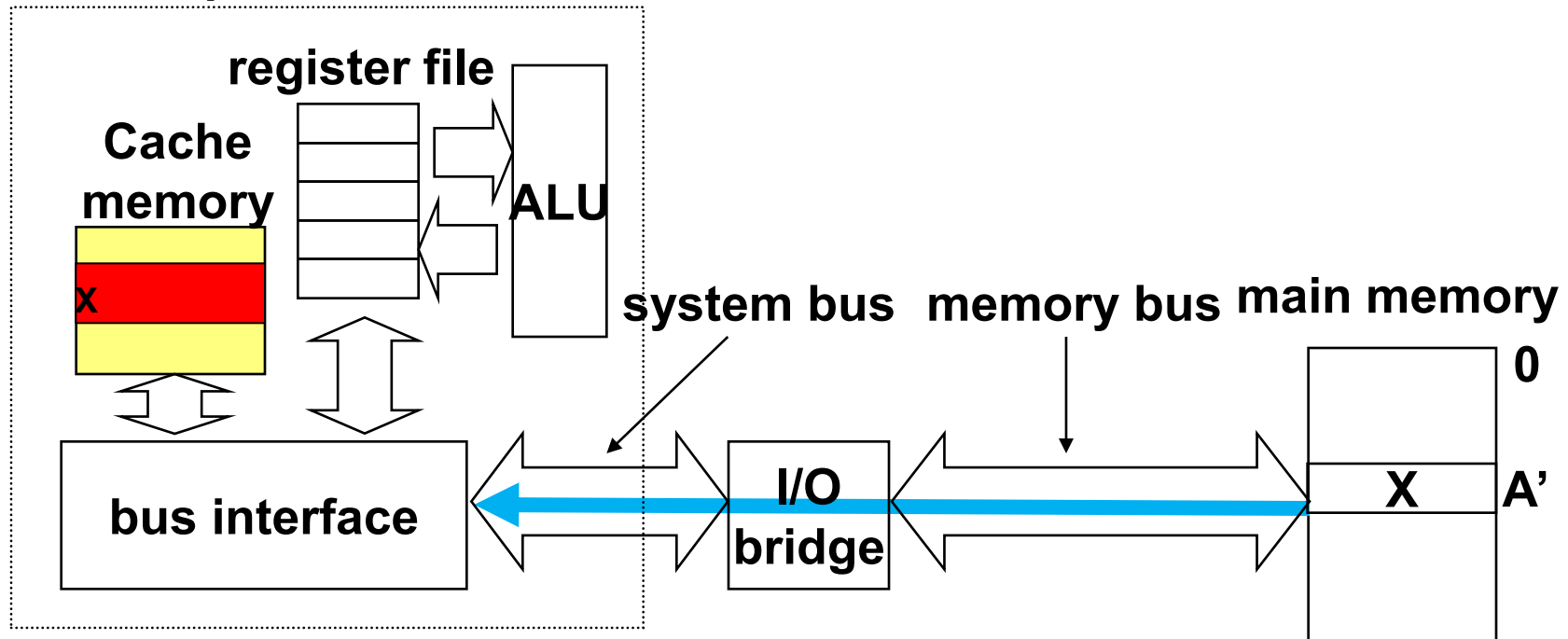
CPU chip



Read the Block from the Memory

CPU reads **x** from the bus and copies it into the cache line

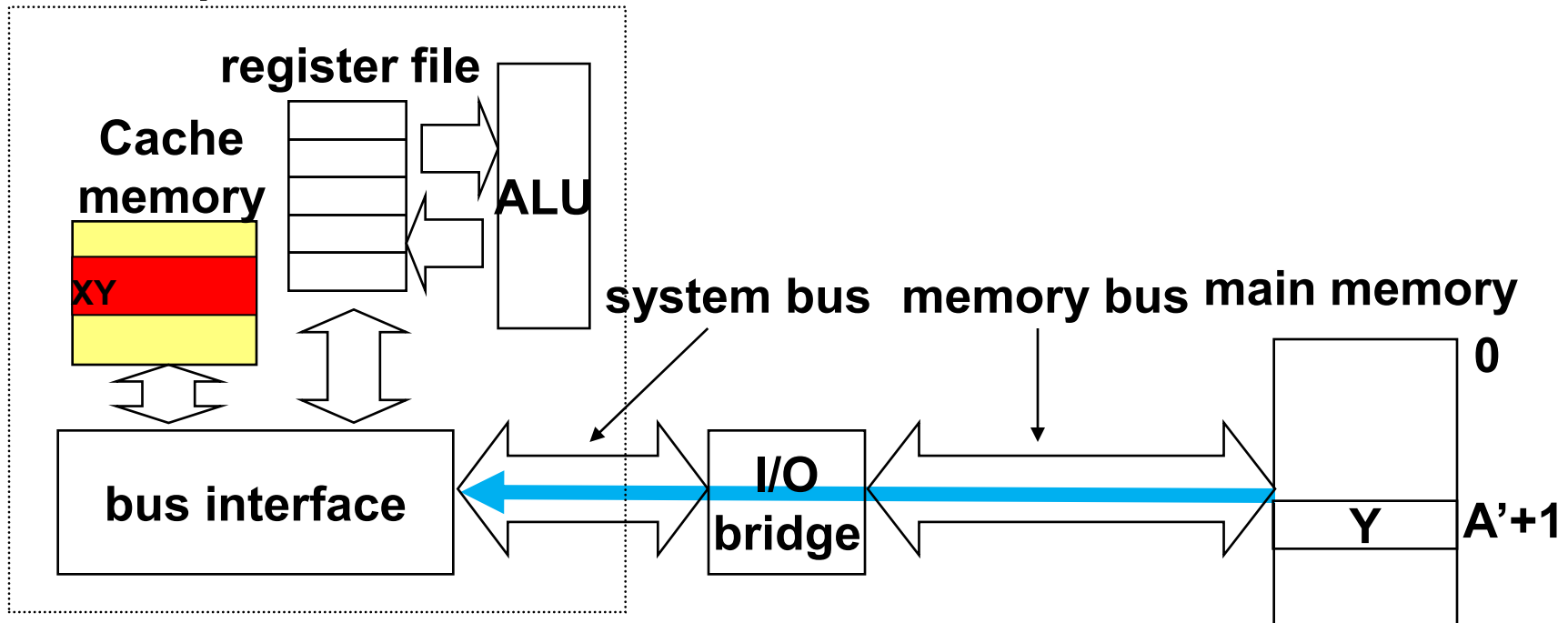
CPU chip



Read the Block from the Memory

Increase A' by 1, and copy Y in $A'+1$ into the cache line.

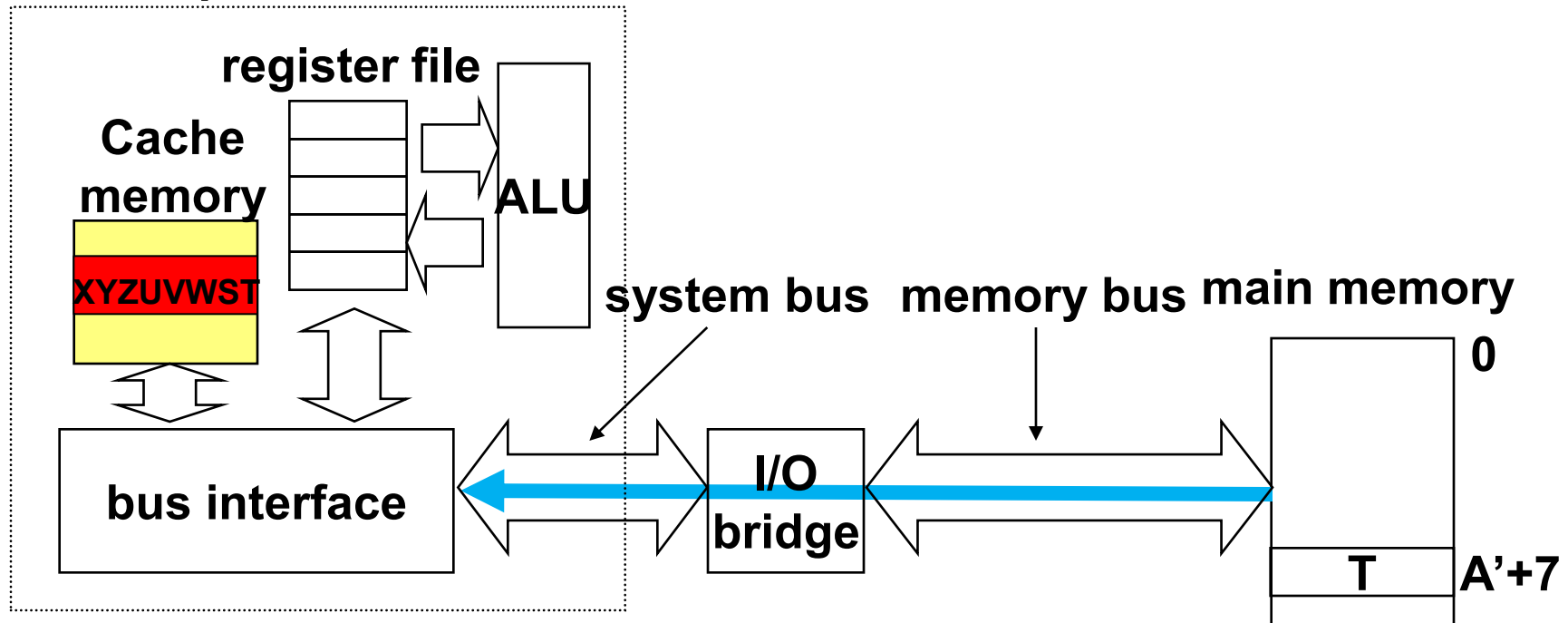
CPU chip



Read the Block from the Memory

Repeat several times (typically 8)

CPU chip



Cache line, set and block

- Block
 - A fixed-sized packet of information
 - Moves back and forth between a cache and main memory (or a lower-level cache)
- Line
 - A container in a cache that stores
 - A block, the valid bit, the tag bits
 - Other information
- Set
 - A collection of one or more lines

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]

set	v	tag	data
0	0		
1	0		
2	0		
3	0		

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]

miss

(1) 0 [0000] (miss)

set v tag data

0	1	0	m[0]m[1]
1	0		
2	0		
3	0		

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

~~0 [0000]~~ 1 [0001] 13 [1101] 8 [1000] 0 [0000]
miss hit

(2) 1 [0001] (hit)

set v tag data

0	1	0	m[0]m[1]
1	0		
2	0		
3	0		

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

~~0 [0000]~~ ~~1 [0001]~~ 13 [1101] 8 [1000] 0 [0000]
~~miss~~ ~~hit~~ miss

(3) 13 [**1**101] (miss)

set v tag data

0	1	0	m[0]m[1]
1	0		
2	1	1	m[12] m[13]
3	0		

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

~~0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]~~
~~miss hit miss miss~~

(4) 8 [1000] (miss)

set v tag data

0	1	1	m[8]m[9]
1	0		
2	1	1	m[12]m[13]
3	0		

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

~~0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]~~
~~miss hit miss miss miss~~

(5) 0 [0000] (miss)

set v tag data

0	1	0	m[0]m[1]
1	0		
2	1	1	m[12]m[13]
3	0		

Direct-mapped cache simulation

Example

M=16 byte addresses

B=2 bytes/block, S=4 sets, E=1 entry/set

t=1	s=2	b=1
x	xx	x

Address trace (reads):

0 [0000]	1 [0001]	13 [1101]	8 [1000]	0 [0000]
miss	hit	miss	miss	miss

Thrashing!

(5) 0 [0000] (miss)

set	v	tag	data
0	1	0	m[0]m[1]
1	0		
2	1	1	m[12]m[13]
3	0		

Conflict Misses in Direct-Mapped Caches

```
1 float dotprod(float x[8], float y[8])
2 {
3     float sum = 0.0;
4     int i;
5
6     for (i = 0; i < 8; i++)
7         sum += x[i] * y[i];
8     return sum;
9 }
```

Conflict Misses in Direct-Mapped Caches

- Assumption for x and y
 - x is loaded into the 32 bytes of contiguous memory starting at address 0
 - y starts immediately after x at address 32
- Assumption for the cache
 - A block is 16 bytes
 - big enough to hold four floats
 - The cache consists of two sets
 - A total cache size of 32 bytes

Conflict Misses in Direct-Mapped Caches

Element	Address	Set index	Element	Address	Set index
x[0]	0	0	y[0]	32	0
x[1]	4	0	y[1]	36	0
x[2]	8	0	y[2]	40	0
x[3]	12	0	y[3]	44	0
x[4]	16	1	y[4]	48	1
x[5]	20	1	y[5]	52	1
x[6]	24	1	y[6]	56	1
x[7]	28	1	y[7]	60	1

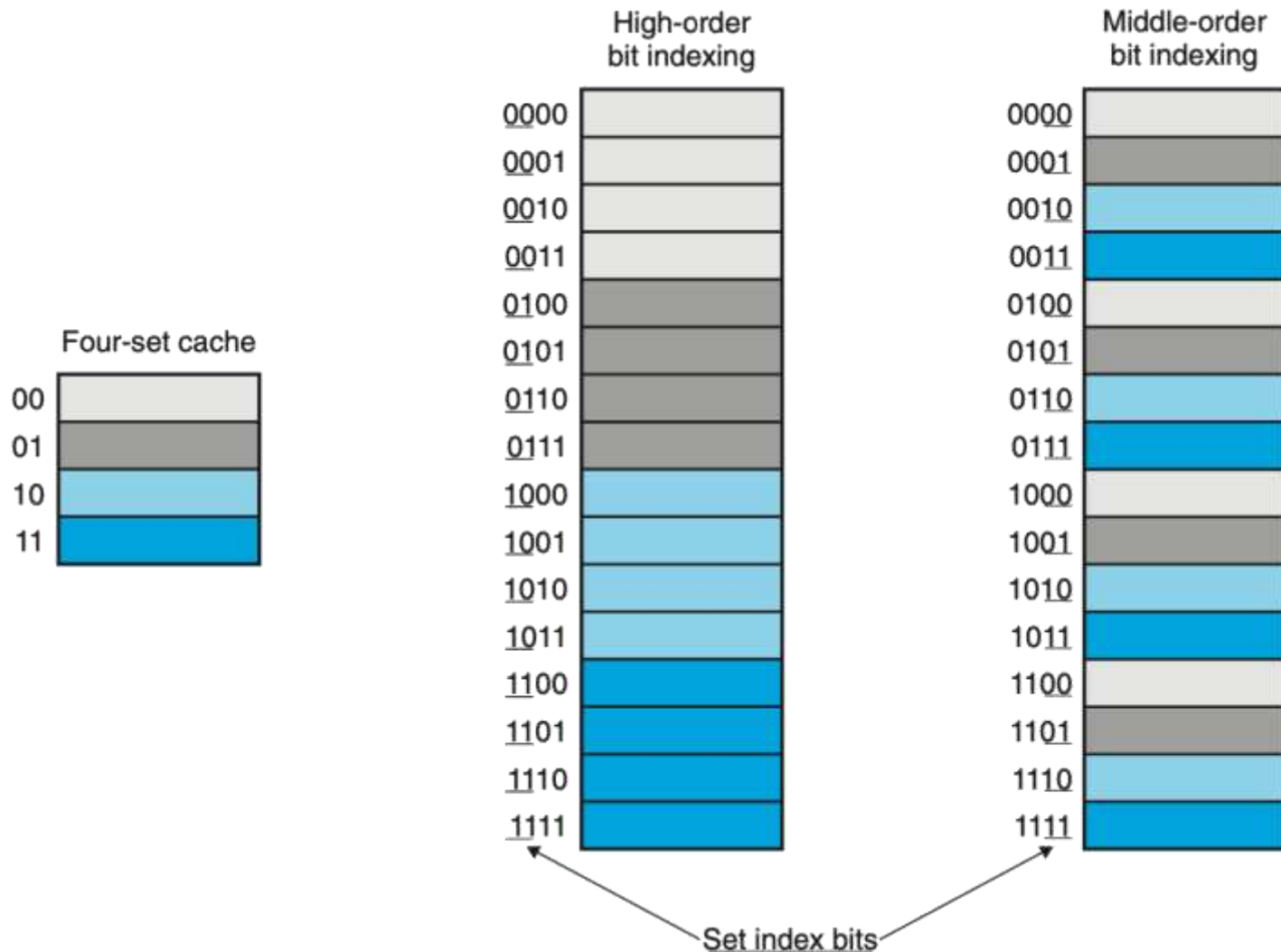
- Trashing
 - Read x[0] will load x[0] ~ x[3] into the cache
 - Read y[0] will overload the cache line by y[0] ~ y[3]

Conflict Misses in Direct-Mapped Caches

Element	Address	Set index	Element	Address	Set index
x[0]	0	0	y[0]	48	1
x[1]	4	0	y[1]	52	1
x[2]	8	0	y[2]	56	1
x[3]	12	0	y[3]	60	1
x[4]	16	1	y[4]	64	0
x[5]	20	1	y[5]	68	0
x[6]	24	1	y[6]	72	0
x[7]	28	1	y[7]	76	0

- Padding can avoid thrashing
 - Claim x[12] instead of x[8]

Why use middle bits as index?

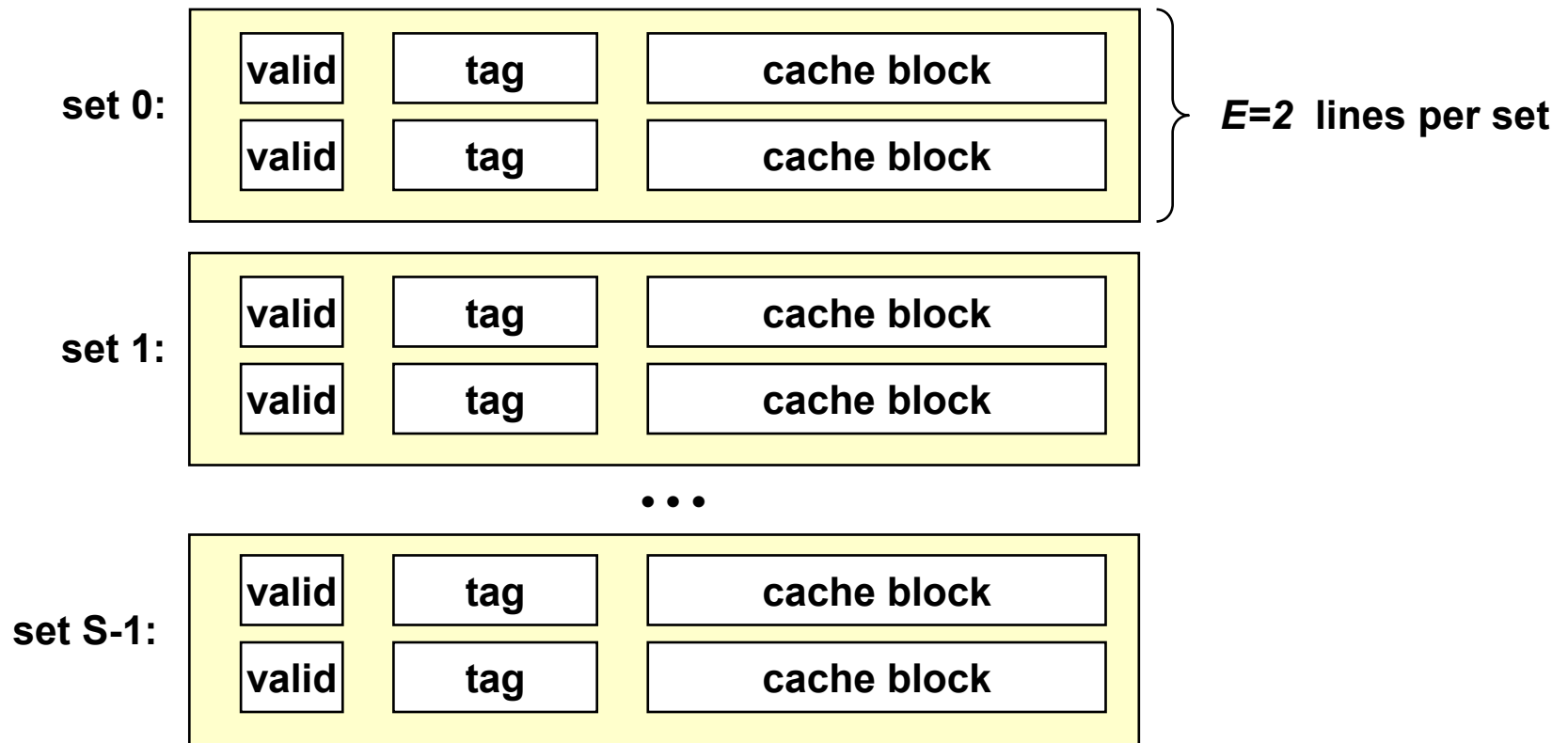


Why use middle bits as index?

- High-Order Bit Indexing
 - Adjacent memory lines would map to same cache entry
 - Poor use of **spatial locality**
- Middle-Order Bit Indexing
 - Consecutive memory lines map to different cache lines
 - Can hold C -byte region of address space in cache at one time

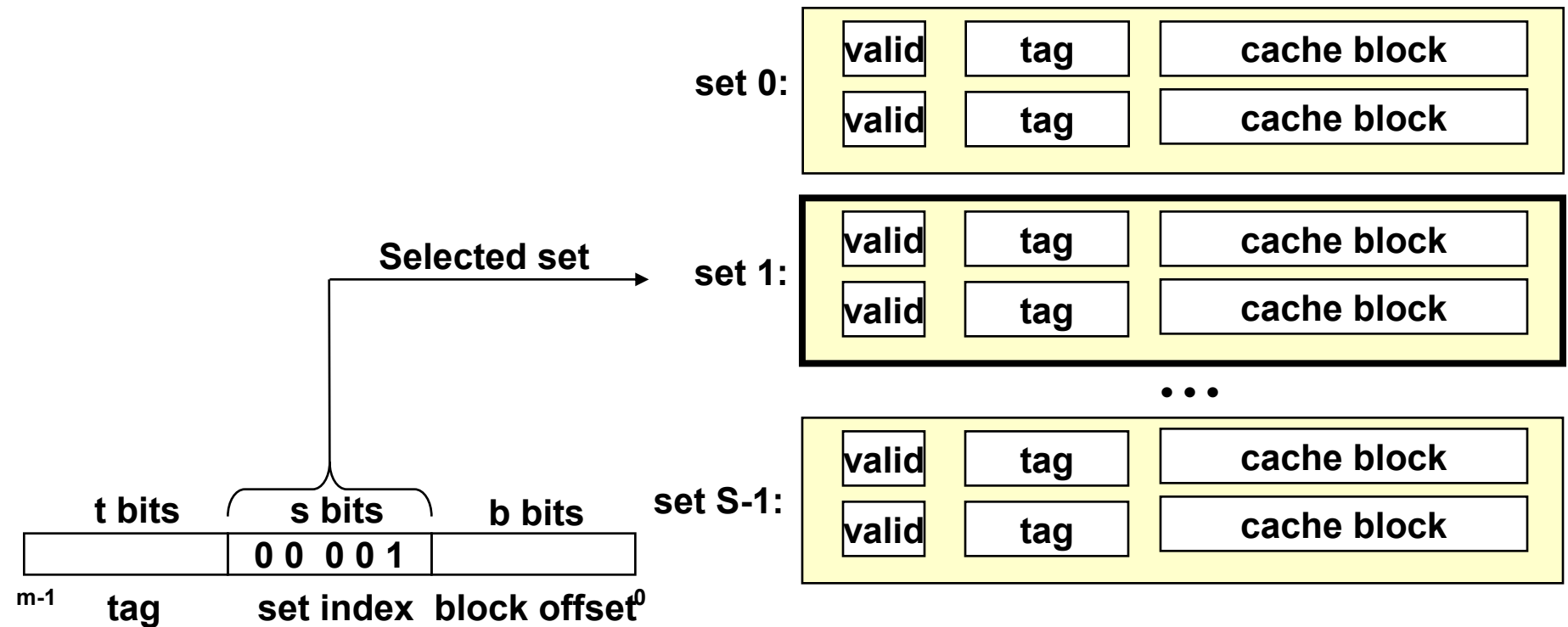
Set associative caches

- Characterized by more than one line per set



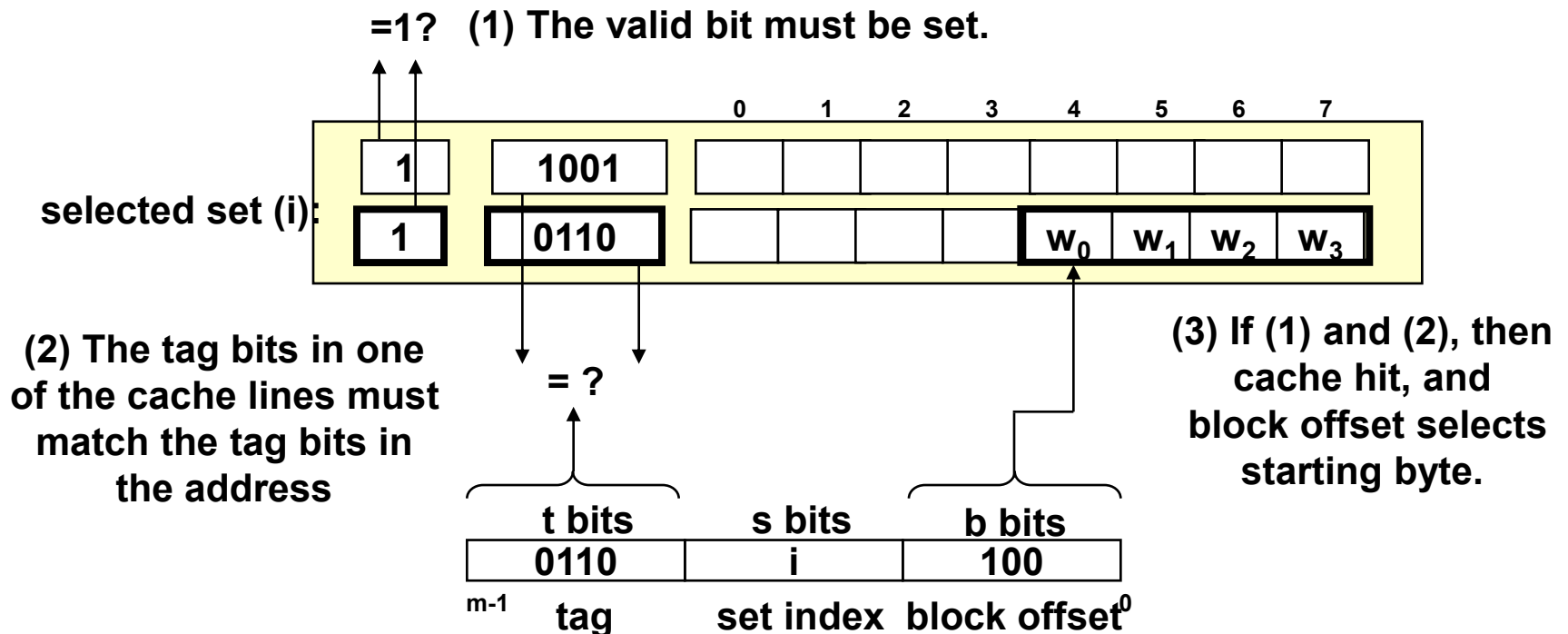
Accessing set associative caches

- Set selection
 - identical to direct-mapped cache



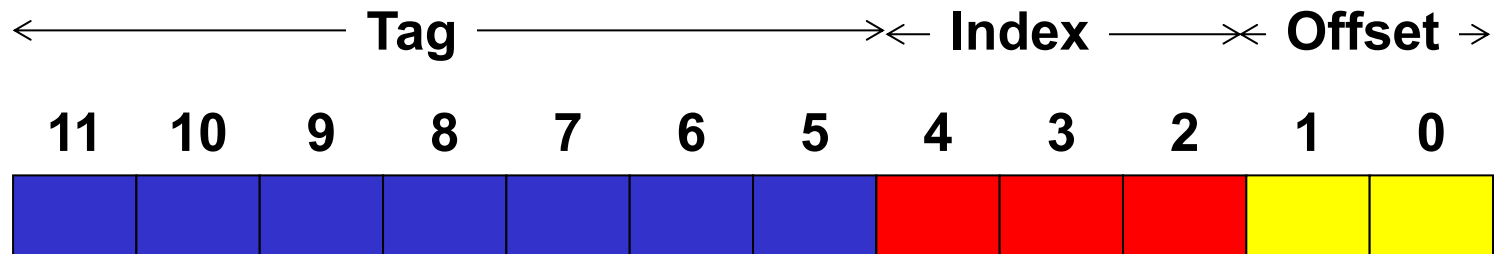
Accessing set associative caches

- Line matching and word selection
 - must compare the tag in each valid line in the selected set



Associative Cache

- Cache
 - 16 lines
 - 4-byte line size
 - 2-way set associative



Simple Memory System Cache

Idx	Tag	Valid	B0	B1	B2	B3
0	19	1	99	11	23	11
0	15	0	-	-	-	-
1	1B	1	00	02	04	08
1	36	0	-	-	-	-
2	32	1	43	6D	8F	09
2	0D	1	36	72	F0	1D
3	31	0	-	-	-	-
3	16	1	11	C2	DF	03

Simple Memory System Cache

Idx	Tag	Valid	B0	B1	B2	B3
4	24	1	3A	00	51	89
4	2D	0	-	-	-	-
5	2D	1	93	15	DA	3B
5	0B	0	-	-	-	-
6	12	0	-	-	-	-
6	16	1	04	96	34	15
7	13	1	83	77	1B	D3
7	14	0	-	-	-	-

Address Translation Example

Address: 0x354

← Tag				Index →				Offset →			
11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	1	0	1	0	1	0	1	0	0

Offset: 0x0

Index: 0x05

Tag: 0x1A

Hit? No

Idx	Tag	Valid	B0	B1	B2	B3
5	2D	1	93	15	DA	3B
5	0B	0	-	-	-	-

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]

set	v	tag	data
0	0		
0	0		
1	0		
1	0		

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]

miss

(1) 0 [0000] (miss)

set v tag data

0	1	00	m[0]m[1]
0	0		
1	0		
1	0		

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

~~0 [0000]~~ 1 [0001] 13 [1101] 8 [1000] 0 [0000]
miss hit

(2) 1 [0001] (hit)

set v tag data

0	1	00	m[0]m[1]
0	0		
1	0		
1	0		

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

~~0 [0000]~~ ~~1 [0001]~~ 13 [1101] 8 [1000] 0 [0000]
miss hit miss

(3) 13 [1101] (miss)

set v tag data

0	1	00	m[0]m[1]
0	1	11	m[12]m[13]
1	0		
1	0		

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

~~0 [0000]~~ ~~1 [0001]~~ ~~13 [1101]~~ ~~8 [1000]~~ 0 [0000]
~~miss~~ ~~hit~~ ~~miss~~ ~~miss~~

(4) 8 [1000] (miss)? LRU

set	v	tag	data
0	1	00	m[0]m[1]
0	1	11	m[12]m[13]
1	1		
1	0		

Line Replacement on Misses

- If all the cache lines of the set are valid
 - Which line is selected to be evicted ?
- LFU (least-frequently-used)
 - Replace the line that has been referenced the fewest times over some past time window
- LRU (least-recently-used)
 - Replace the line that was last accessed the furthest in the past
- All of these policies require additional time and hardware

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

~~0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]~~
~~miss hit miss miss~~

(4) 8 [1000] (miss)

set v tag data

0	1	10	m[8]m[9]
0	1	11	m[12]m[13]
1	1		
1	0		

Set Associative Cache Simulation

Example

M=16 byte addresses

B=2 bytes/block, S=2 sets, E=2 entry/set

t=2	s=1	b=1
xx	x	x

Address trace (reads):

~~0 [0000] 1 [0001] 13 [1101] 8 [1000] 0 [0000]~~
~~miss hit miss miss miss~~

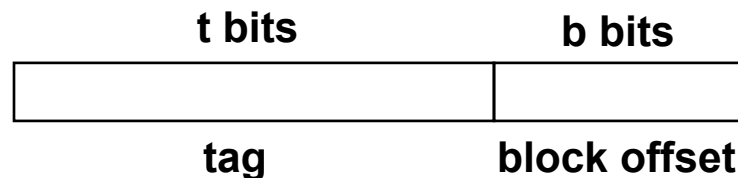
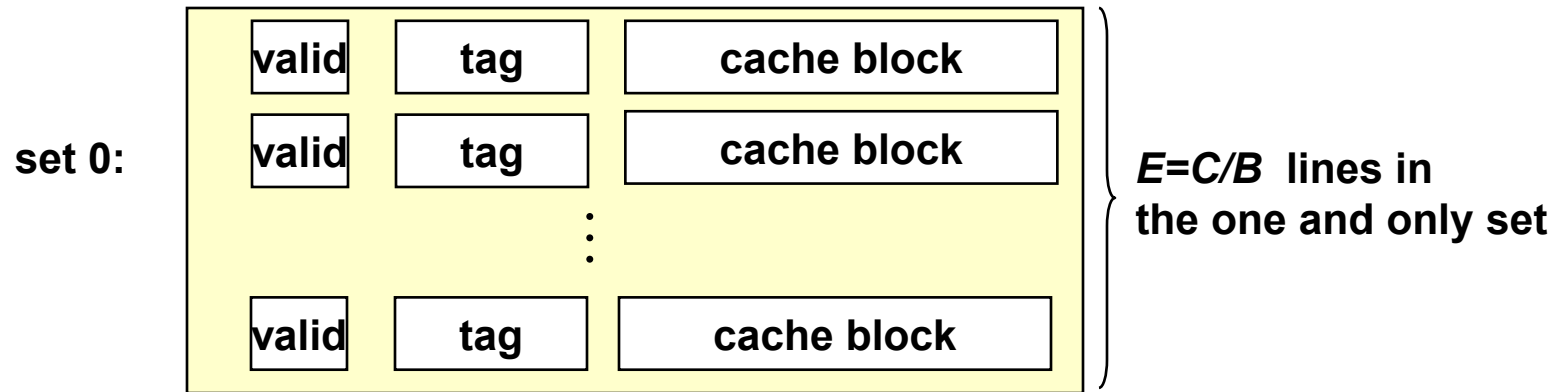
(5) 0 [0000] (miss) LRU

set v tag data

0	1	10	m[8]m[9]
0	1	00	m[0]m[1]
1	1		
1	0		

Fully associative caches

- Characterized by all of the lines in the only one set
- No set index bits in the address



Accessing fully associative caches

- Line match and Word selection
 - must compare the tag in each valid line

